

HTB ACADEMY

Splunk for Security Analysts

HTB Academy · SIEM / SOC Analyst Track · Study Notes

A complete illustrated walkthrough of two Sysmon-driven Splunk investigations — an LSASS credential-dumping chain and a process-injection anomaly hunt — written entirely in SPL. Lab answers are partially masked as a hint.

Platform	HackTheBox Academy
Focus	Splunk SPL for SOC investigation
Data source	Sysmon (Event IDs 7, 8, 10) ingested into Splunk
Analyst	Chayan Panchal · @tron874
Core Skills	SPL, stats/eventstats, Sysmon telemetry, anomaly detection

0 Quick Reference — Read This First



SPL & Sysmon quick reference

```
EventCode=10    ProcessAccess (LSASS dumping, credential access)
EventCode=7     ImageLoaded (DLL loads – comsvcs.dll, clr.dll, etc.)
EventCode=8     CreateRemoteThread (process injection)
EventCode=3     Network connection (C2 / DestinationIp)

| stats count by <field>                                # simple aggregation
| eventstats avg(x) AS avg, stdev(x) AS stdev            # add stats without collapsing rows
| where x > avg + (2*stdev)                              # statistical outlier filter
| table field1 field2 ...                                # project specific columns
```

1 Investigation 1 — Who Dumped LSASS?

Sysmon Event ID 10 (ProcessAccess) logs any time one process opens a handle into another's memory — exactly what happens when a credential dumping tool reads LSASS. Searching for that event against the lsass process surfaces every process that touched it.



SPL — find the process that accessed LSASS

```
index="main" EventCode=10 lsass | stats count by SourceImage
```

```
Process that dumped lsass -> r.....3.....e
```

NOTE rundll32.exe accessing lsass.exe directly is unusual — rundll32 is meant to load and run functions from a DLL, not touch another process's memory. That mismatch is the first red flag.



SPL — which DLL did rundll32 load to do it?

```
index="main" EventCode=7 Image="*rundll32.exe" | stats count by ImageLoaded
```

```
Misused DLL -> c.....c.....l
```

TECHNIQUE comsvcs.dll exports a MiniDump function never meant to be called this way — invoking it via rundll32 against lsass.exe's PID is a well-known, living-off-the-land technique for dumping credentials without ever touching disk with a dedicated tool like Mimikatz.

2 Investigation 1 — Tracing the C2 Channel

With the dumping technique confirmed, the next step looks for a related execution technique — loading the .NET CLR is a strong indicator of an execute-assembly / C# in-memory loader, a common way post-exploitation frameworks run code without dropping a binary.

```
SPL — find suspicious .NET CLR loads

index="main" EventCode=7 ImageLoaded="*clr.dll" | stats count by Image | sort -count

Process suspicious clr.dll loads trace back to -> r.....3.....e
```

```
SPL — network connections from the compromised process

# Pivot to network telemetry for that same process
index=* EventCode=3 Image="*rundll32.exe" | stats count by DestinationIp | sort -count

C2 callback server IPs -> 1.....n.....1
```

NOTE This is the shape of a real intrusion chain reconstructed entirely from Sysmon telemetry: credential access (Event 10) -> in-memory code execution (Event 7, clr.dll) -> command-and-control over the network (Event 3) — three different Sysmon event types tied together by one process image.

3 Investigation 2 — Hunting Process Injection Statistically

The second investigation doesn't start from a known bad process — it starts from Sysmon Event ID 8 (CreateRemoteThread) across the whole environment, and uses statistics to find what looks abnormal rather than searching for a specific known indicator.



SPL — statistical outlier detection for thread injection

```
index=* EventCode=8
| stats count AS injected_threads BY SourceImage
| eventstats avg(injected_threads) AS avg, stdev(injected_threads) AS stdev
| where injected_threads > avg + (2*stdev)
| sort -injected_threads
```



Result (masked)

```
Outlier process (>2 standard deviations above average) -> r.....i.....e
```

TECHNIQUE eventstats adds the average and standard deviation as new fields on every row without collapsing the result set — which is what makes the where `injected_threads > avg + (2*stdev)` filter possible. This is a real anomaly-detection pattern, not just a keyword search — useful for catching injection techniques that don't match any known signature.



SPL — confirming the injection target

```
index=* rundll32.exe
| search EventCode=8
| table _time SourceImage TargetImage SourceProcessId TargetProcessId StartAddress StartModule
```

NOTE The outlier process from the statistical search is confirmed here as the source of repeated CreateRemoteThread events targeting the same process identified in Investigation 1 — tying both investigations to a single injection technique used twice in the environment.